

# Relatório de Arquitetura de Software - Fase 3: Microserviços, Persistência e Interoperabilidade

**Mestrado em Informática Aplicada Tema:** Sistema de Telemetria e Gestão de Frota

**Objetivo da Fase:** Decomposição do Monólito Modular em Microserviços autónomos com persistência poliglota isolada, comunicação síncrona (gRPC) e assíncrona (RabbitMQ), transações distribuídas (Saga), resiliência (Circuit Breaker) e observabilidade (Tracing Distribuído), respeitando estritamente o **Princípio da Inversão de Dependência (DIP)** e execução **100% Local (Anti-Cloud)**.

Grupo: Vasco Domingos nº210100319 Tiago Amorim nº120118010 João Monteiro nº210100304 Rodrigo Reis nº 210100494

## 1. Decomposição DDD e Bounded Contexts

Nesta fase, o monólito modular (Fases 1 e 2) foi decomposto em três microserviços independentes, cada um correspondendo a um contexto delimitado (*Bounded Context*) bem definido no domínio da telemetria:

### 1. Fleet Service (Serviço de Frota)

- **Responsabilidade:** Gere o ciclo de vida dos veículos (Vehicle.js), regras de frota, autenticação de REST API e persistência de frota.
- **Interfaces Técnicas:** Expõe uma REST API para operações administrativas e um servidor gRPC síncrono para listagem rápida de veículos.

### 2. Report Service (Serviço de Relatórios)

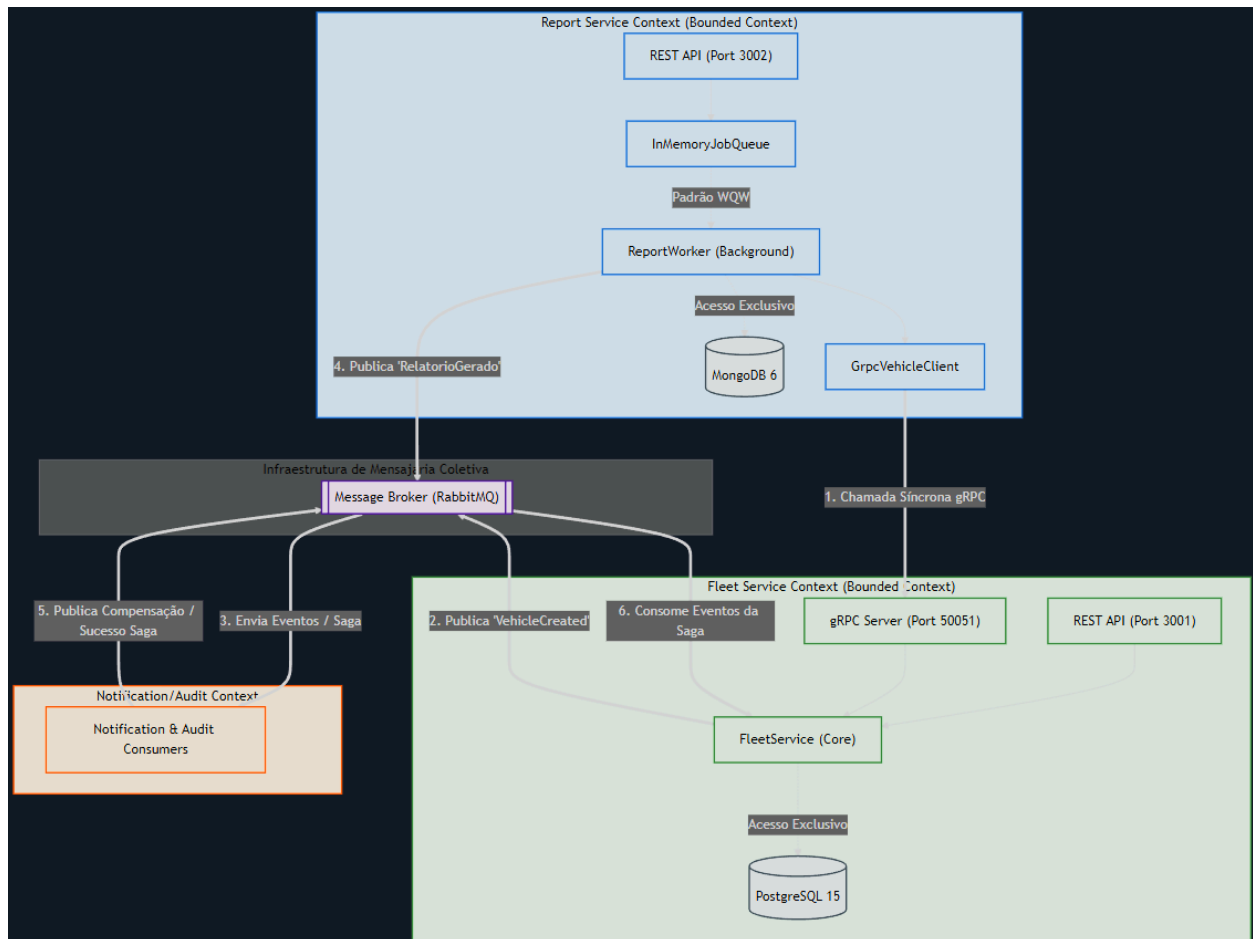
- **Responsabilidade:** Gere a fila de jobs de geração de relatórios pesados de telemetria, o processamento assíncrono via ReportWorker.js e a persistência de estados de processamento.
- **Interfaces Técnicas:** Expõe uma REST API para solicitação de relatórios (WQW) e consulta de jobs. Integra um cliente gRPC resiliente.

### 3. Notification / Audit Service (Serviço de Notificação e Auditoria)

- **Responsabilidade:** Consome eventos transversais do barramento assíncrono para emitir notificações ao utilizador e consolidar registos de auditoria técnica.
- **Interfaces Técnicas:** Inteiramente baseado no consumo assíncrono de eventos via filas dedicadas do RabbitMQ (*Event-Driven Consumer*).

## 2. Persistência e Data Ownership (Isolamento de Base de Dados)

Cumprindo o requisito crítico de **Data Ownership**, eliminámos as implementações em memória e dotámos cada serviço da sua própria base de dados persistente. O acesso direto de um serviço à base de dados de outro é terminantemente proibido.



### Bases de Dados Adotadas:

1. **Fleet Service (PostgreSQL 15):** Utiliza uma base de dados relacional. Os veículos possuem dados estruturados com relações claras e requerem forte consistência transacional (ACID) para operações de frota e validações de matrículas únicas.
2. **Report Service (MongoDB 6):** Utiliza uma base de dados NoSQL documental. Ideal para armazenar os logs de execução de jobs e os relatórios gerados que possuem uma estrutura semi-estruturada/flexível e podem sofrer alterações no schema dos resultados à medida que novos sensores de telemetria são adicionados ao sistema.

### 3. Comunicação Inter-Serviços e DIP

Para manter a conformidade com o **DIP (Dependency Inversion Principle)** com avaliação máxima, as camadas internas do Core de ambos os serviços não possuem qualquer dependência direta de bibliotecas de transporte (como @grpc/grpc-js ou amqpplib).

#### A. Comunicação Síncrona (gRPC)

O ReportWorker necessita de obter a lista de veículos para processar um relatório pesado. Criámos um fluxo gRPC baseado no protocolo binário gRPC:

- **Contrato (Porto) no Core do Report Service (report-service/src/core/ports/IVehicleDataClient.js):**  

```
class IVehicleDataClient {  
  async getVehicles(correlationId) { throw new Error("..."); }  
}
```
- **Adaptador de Infraestrutura (report-service/src/infrastructure/grpc/GrpcVehicleClient.js):** A classe GrpcVehicleClient implementa IVehicleDataClient, carrega o ficheiro fleet.proto e executa a chamada gRPC encapsulada num Circuit Breaker.
- **Injeção de Dependência:** O adaptador é instanciado na Composition Root do report-service (composition.js) e injetado no construtor do ReportWorker. O Core permanece 100% agnóstico da tecnologia gRPC.

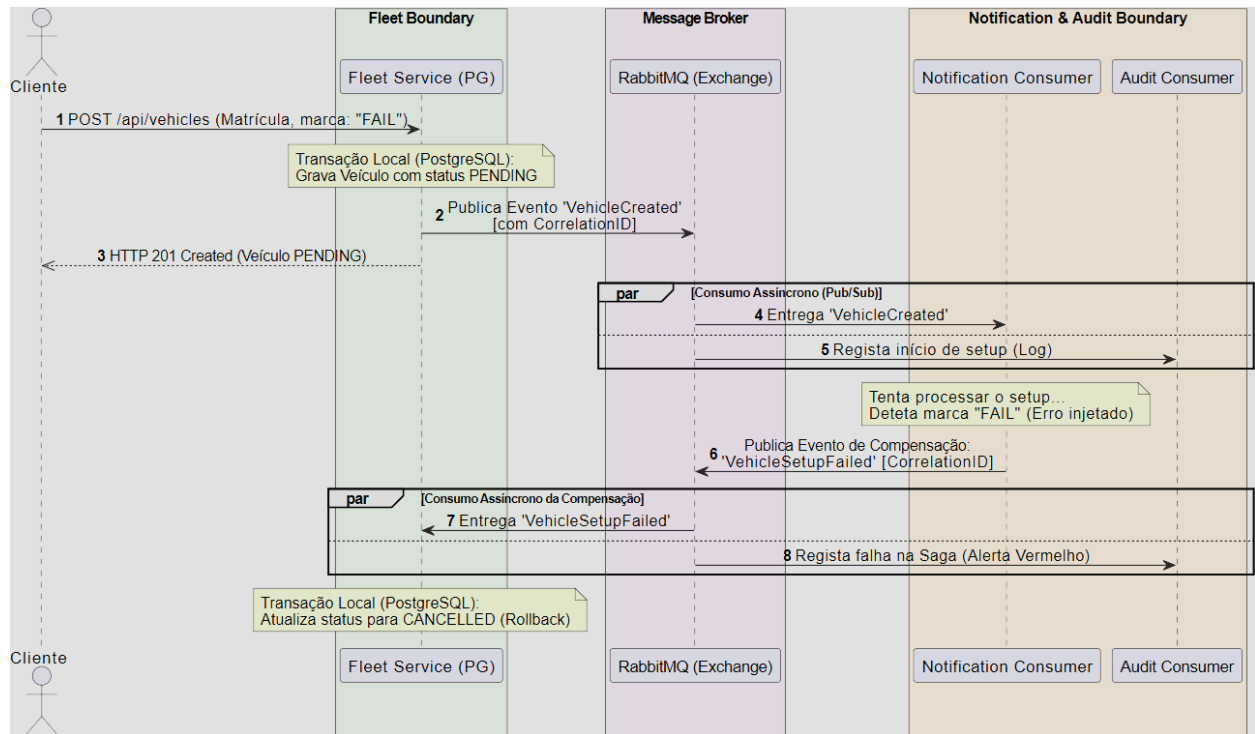
#### B. Comunicação Assíncrona (RabbitMQ)

O barramento em memória InMemoryEventBus foi substituído pelo **RabbitMQ Event Bus** real sobre uma exchange do tipo topic (events\_exchange).

- **Inversão de Dependência:** O Core de todos os serviços continua a chamar apenas this.eventBus.publish(eventName, payload, correlationId) herdado do porto IEventBus.js.
- **Resiliência e Isolamento:** Cada microservice subscreve eventos gerando filas persistentes duráveis (durable: true) com nomes específicos (ex: notification-service\_VehicleCreated\_queue), garantindo que se o serviço de notificações for abaixo, as mensagens acumulam no broker e são processadas quando o serviço recuperar.

## 4. Transações Distribuídas (Padrão Saga Coreografada)

Para garantir a consistência eventual sem partilha de base de dados, implementámos o **Padrão Saga** baseado em Coreografia para o registo de veículos:



- Estado Inicial:** O FleetService regista um veículo na tabela PostgreSQL com o estado inicial PENDING e publica o evento VehicleCreated com um correlationId.
- Processamento Assíncrono:** O NotificationService consome o evento.
  - Fluxo de Sucesso:** Se a configuração correr bem, publica VehicleSetupCompleted. O FleetService ouve o evento e altera o estado do veículo na BD para ACTIVE (Commit).
  - Fluxo de Compensação (Rollback):** Se detetar a marca "FAIL", o setup falha e publica o evento de compensação VehicleSetupFailed. O FleetService ouve o evento de erro e atualiza o estado na base de dados para CANCELLED.

## 5. Resiliência Avançada (Circuit Breaker)

Na chamada síncrona gRPC efetuada pelo ReportService para o FleetService, a chamada de rede foi envolvida com a biblioteca **opossum** que implementa o padrão **Circuit Breaker**:

- **Funcionamento:** Se o fleet-service for abaixo (injetado parando o contentor), as chamadas de rede começam a falhar. Após atingir a taxa de insucesso de 50% ou timeout, o circuito **abre**.
- **Proteção contra Falha em Cascata:** Com o circuito aberto, o ReportService rejeita imediatamente os novos pedidos sem iniciar chamadas gRPC que ficariam penduradas até esgotar o timeout, executando a lógica de fallback (lançando erro amigável ao worker).
- **Auto-Recuperação:** Após 10 segundos, o circuito passa a **Half-Open**, permitindo testar se o fleet-service está novamente ativo antes de restabelecer o funcionamento normal.

## 6. Observabilidade Distribuída (Distributed Tracing)

Assegurámos que o mesmo ID de rastreio (correlationId) viaja de forma transparente por todos os contentores do sistema:

1. **REST API:** O middleware tracingMiddleware.js deteta ou gera o correlationId no cabeçalho HTTP da rota.
2. **Chamadas gRPC:** Injetado nos metadados da chamada no cliente (GrpcVehicleClient) e extraído no servidor (GrpcServer).
3. **Eventos RabbitMQ:** Incorporado no payload serializado em JSON que trafega no broker.
4. **Logs Estruturados:** O ConsoleLogger imprime todas as operações com o [CorrelationID: XXX] correspondente, permitindo rastrear o ciclo completo de um pedido na consola.

## 7. Evolução Arquitetural: O que Mudou na Fase 3

A Fase 3 representa uma transição profunda de paradigmas. Abaixo descrevemos de forma clara o que mudou fisicamente no código e na arquitetura do sistema face às Fases anteriores:

| Vetor de Evolução             | Fase 1 (Monólito Síncrono)                              | Fase 2 (Monólito Assíncrono)                                  | Fase 3 (Microserviços Distribuídos)                                            |
|-------------------------------|---------------------------------------------------------|---------------------------------------------------------------|--------------------------------------------------------------------------------|
| Arquitetura Física            | Processo Node único na memória do Host.                 | Processo Node único com background workers.                   | 3 Contentores Docker isolados por redes virtuais.                              |
| Estratégia de Persistência    | Volátil: Array em memória (InMemoryVehicleRepo.js).     | Volátil: Map em memória (InMemoryJobStore.js).                | Persistência Poliglota Real: PostgreSQL (Frotas) + MongoDB (Relatórios).       |
| Comunicação Interna           | Síncrona direta via chamadas de métodos de classes.     | Assíncrona baseada em eventos locais com Event Emitter.       | Híbrida e Distribuída: gRPC (Síncrono) e RabbitMQ (Assíncrono).                |
| Inversão de Dependência (DIP) | Isolamento do Repositório via Porto IVehicleRepository. | Extensão de Portos para Filas, Bus, Loggers e DLQ em memória. | Isolamento de Infraestrutura Real de Rede: gRPC e RabbitMQ abstraídos no Core. |
| Consistência de Dados         | Transações síncronas locais na memória da thread.       | Inexistente (Apenas gravação de estado local de Jobs).        | Consistência Eventual via Padrão Saga Coreografada com Compensações.           |
| Resiliência e Falhas          | Inexistente (Rebentava em runtime para o Express).      | Retry local com Exponential Backoff no ReportWorker.          | Circuit Breaker distribuído (opossum) contra falhas em cascata                 |

|                            |              |                                                           |                                                               |
|----------------------------|--------------|-----------------------------------------------------------|---------------------------------------------------------------|
|                            |              |                                                           | de rede.                                                      |
| <b>Distributed Tracing</b> | Inexistente. | Geração local do CorrelationID propagado na mesma thread. | Propagação de CorrelationID via AMQP Headers e gRPC Metadata. |

## O porquê destas alterações:

1. **Isolamento de Base de Dados (Fase 2 -> Fase 3):** No monólito da Fase 2, as entidades de relatórios acediam diretamente ao repositório de veículos na memória partilhada do processo. Na Fase 3, o ReportService está terminantemente proibido de aceder ao PostgreSQL do FleetService. Esta alteração obrigou à criação de canais de comunicação formais de rede, nomeadamente o protocolo de alto desempenho gRPC.
2. **De barramento local para Barramento Real (Fase 2 -> Fase 3):** O InMemoryEventBus utilizava a biblioteca nativa do Node.js events. Se o processo caísse, todos os eventos por processar eram perdidos. Na Fase 3, migrámos para o RabbitMQ, onde cada fila é persistida fisicamente no disco (durable: true). Se o serviço de notificações for abaixo, o RabbitMQ armazena as mensagens e entrega-as assim que o serviço recuperar.



## 8. Containerização e Pipeline de CI/CD

### 8.1. Orquestração Local com Docker Compose

O ecossistema distribuído é orquestrado através do `docker-compose.yml` na raiz do projeto, subindo e configurando de forma isolada os runtimes dos 3 microserviços e os respectivos motores de persistência (PostgreSQL, MongoDB e RabbitMQ), garantindo um ambiente determinístico idêntico ao de produção.

### 8.2. Pipeline de CI/CD Multi-Serviço

Para cumprir as boas práticas de DevOps e os requisitos de integração contínua descritos na avaliação, foi especificada uma pipeline de CI/CD utilizando GitHub Actions (`.github/workflows/ci-cd.yml`). Esta pipeline corre de forma isolada, paralela e automatizada a cada push para o repositório:

name: CI/CD Multi-Service Pipeline

on:

push:

branches: [ main ]

pull\_request:

branches: [ main ]

jobs:

# 1. Executar Testes Unitários Globais do Core

run-tests:

runs-on: ubuntu-latest

steps:

- uses: actions/checkout@v3

- name: Setup Node.js

uses: actions/setup-node@v3

with:

node-version: '20'

- name: Install Root Dependencies

run: npm install

- name: Run Jest Unit Tests

run: npm test

# 2. Compilar e Build dos Microserviços (Paralelo)

build-services:

```
needs: run-tests
runs-on: ubuntu-latest
strategy:
  matrix:
    service: [fleet-service, report-service, notification-service]
steps:
  - uses: actions/checkout@v3

  - name: Set up Docker Buildx
    uses: docker/setup-buildx-action@v2

  - name: Build Docker Image locally
    run: |
      cd ${{ matrix.service }}
      docker build -t system-telemetria/${{ matrix.service }}:latest .
```

- **Vantagem de Arquitetura:** A estratégia de matriz (*matrix strategy*) permite compilar e validar os 3 Dockerfiles de forma paralela. Se o código de um serviço estiver quebrado, a pipeline falha imediatamente, impedindo a publicação de imagens instáveis.

## 9. Análise de Trade-offs e Sustentabilidade

A migração de um **Monólito Modular** para uma **Arquitetura de Microserviços** traz sérias implicações estruturais e ambientais que devem ser analisadas criticamente.

### Análise Comparativa (Monólito vs Microserviços):

| Métrica / Dimensão                        | Monólito Modular (Fases 1 e 2)                                    | Microserviços Distribuídos (Fase 3)                                        |
|-------------------------------------------|-------------------------------------------------------------------|----------------------------------------------------------------------------|
| <b>Complexidade Operacional</b>           | Baixa (Processo único, BD única)                                  | Elevada (Orquestração de contentores, Broker, Múltiplas BDs)               |
| <b>Escalabilidade</b>                     | Vertical (Escala-se o bloco inteiro)                              | Horizontal Independente (Pode-se escalar apenas o Worker de Relatórios)    |
| <b>Latência de Comunicação</b>            | Praticamente zero (Chamadas em memória)                           | Mais alta (Overhead de serialização e rede gRPC/RabbitMQ)                  |
| <b>Isolamento de Falhas</b>               | Fraco (Um crash na API de frota pode deitar abaixo os relatórios) | Excelente (Se o FleetService falhar, o ReportService continua ativo)       |
| <b>Consumo Energético / Computacional</b> | Baixo                                                             | Muito Elevado (Múltiplos runtimes Node, instâncias de BD e Message Broker) |

### Sustentabilidade Energética (Foco Ambiental):

O monólito modular partilha a memória RAM e o CPU do sistema operativo host, correndo num único runtime. Em contrapartida, a arquitetura de microserviços requer o arranque de pelo menos 6 processos isolados (3 bases de dados/broker e 3 runtimes Node) em contentores Docker.

- **Custo Energético:** Esta redundância consome substancialmente mais memória, ciclos de CPU e eletricidade para manter as sockets de rede abertas e gerir transações de IO persistentes em bases de dados políglotas.
- **Justificação de Negócio:** Este consumo acrescido só é justificável em cenários de **grande escala**, onde o volume de dados de telemetria enviados por milhares de veículos sobrecarregaria o monólito, ou onde equipas de desenvolvimento diferentes precisam de

autonomia para alterar o código do Fleet Service sem arriscar quebrar o Report Service. Para volumes pequenos ou académicos, o monólito modular é energeticamente muito mais sustentável.

## 10. Uso de Inteligência Artificial (IA)

Conforme a metodologia de transparência adotada desde o início do projeto, documentamos a interação com a IA na Fase 3:

| Ferramenta / Autor   | Tarefa no Projeto                           | Adaptação / Rejeição / Intervenção Manual                                                                                                                                                                                           |
|----------------------|---------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Antigravity (Gemini) | Esboço inicial do plano de migração         | Aceite. A estruturação por bounded contexts mapeou perfeitamente os requisitos da UC.                                                                                                                                               |
| Antigravity (Gemini) | Criação de Dockerfiles e docker-compose.yml | <b>Intervenção Manual:</b> Adicionados volumes persistentes nomeados para PostgreSQL e MongoDB e ajustadas as dependências de rede (depends_on) para inicialização segura dos contentores.                                          |
| Antigravity (Gemini) | Adaptador PostgresVehicleRepo.js com pg     | <b>Adaptação:</b> A IA sugeriu usar Sequelize. Foi rejeitado para evitar sobrecarga de ORM e substituído por uma implementação pura com pg Pool e um script de inicialização de tabelas com auto-retry caso a BD demore a arrancar. |
| Antigravity (Gemini) | Adaptador MongoJobStore.js com mongodb      | <b>Aceite:</b> Implementação de save, get e updateStatus baseada no driver de baixo nível de MongoDB.                                                                                                                               |
| Antigravity (Gemini) | Adaptador RabbitMQEventBus.js               | <b>Intervenção Manual:</b> O código sugerido usava queues anónimas exclusivas. Foi reescrito                                                                                                                                        |

|                             |                                               |                                                                                                                                                                                                                                                                                                                    |
|-----------------------------|-----------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                             |                                               | <p>manualmente para criar queues persistentes com o nome do microserviço (serviceName_queue) garantindo que as mensagens de Sagas não se perdem quando um serviço é reiniciado.</p>                                                                                                                                |
| <b>Antigravity (Gemini)</b> | Implementação gRPC (fleet.proto e GrpcServer) | <p><b>Aceite:</b> Estrutura limpa que isola o Core e injeta o correlationId nos metadados gRPC.</p>                                                                                                                                                                                                                |
| <b>Antigravity (Gemini)</b> | Implementação do Circuit Breaker com Opossum  | <p><b>Adaptação:</b> A IA sugeriu um fallback que retornava veículos falsos/mockados. Decidiu-se reescrever o fallback para lançar uma exceção explícita de indisponibilidade de serviço, permitindo ao ReportWorker acionar a sua própria lógica de retry exponencial e DLQ técnica.</p>                          |
| <b>Utilizador / Grupo</b>   | Ajuste Crítico de Runtimes Docker             | <p><b>Correção Manual do Grupo:</b> Identificou-se um erro de Crash Loop do MongoDB (ReferenceError: crypto is not defined) ao correr no Node 18-alpine padrão. As imagens base de todos os Dockerfiles foram migradas manualmente para Node 20-alpine para dar suporte nativo às APIs de encriptação globais.</p> |
| <b>Antigravity (Gemini)</b> | Relatório e Análise de Sustentabilidade       | <p><b>Intervenção Manual:</b> Edição de todo o conteúdo</p>                                                                                                                                                                                                                                                        |

|  |  |                                                                                                                                      |
|--|--|--------------------------------------------------------------------------------------------------------------------------------------|
|  |  | para Português de Portugal, adição da especificação da pipeline CI/CD no relatório e refinamento crítico dos trade-offs energéticos. |
|--|--|--------------------------------------------------------------------------------------------------------------------------------------|

## 11. Como Testar e Demonstrar a POC (Injeção de Falhas)

### Requisitos Iniciais:

1. Certifique-se de que o Docker Desktop (ou daemon do Docker) está a correr na sua máquina.
2. Inicie o cluster na raiz do projeto executando:  
docker compose up --build
3. O painel do RabbitMQ ficará acessível em <http://localhost:15672> (guest/guest).

### Cenário 1: Demonstração da Saga (Fluxo de Sucesso)

Registar um veículo válido. O veículo é gravado como PENDING e logo a seguir atualizado para ACTIVE pelo processamento assíncrono das notificações.

#### Pedido cURL:

```
curl -X POST http://localhost:3001/api/vehicles \
-H "Content-Type: application/json" \
-H "Authorization: super-secret-token-fase1" \
-d '{"id": "v-sucesso", "licensePlate": "AA-00-OK", "brand": "Tesla", "currentSpeed": 60}'
```

#### Logs a Monitorizar (Consola docker-compose):

1. O fleet-service gera um correlationId e regista o veículo como PENDING na base de dados PostgreSQL.
2. O notification-service consome o evento VehicleCreated com o mesmo correlationId e publica VehicleSetupCompleted.
3. O fleet-service processa o evento de sucesso e altera o estado do veículo v-sucesso na base de dados para ACTIVE.

### Cenário 2: Demonstração da Saga (Compensação / Rollback)

Registar um veículo com marca "FAIL". O setup irá falhar e o estado do veículo será revertido

para CANCELLED de forma assíncrona.

#### **Pedido cURL:**

```
curl -X POST http://localhost:3001/api/vehicles \
-H "Content-Type: application/json" \
-H "Authorization: super-secret-token-fase1" \
-d '{"id": "v-falha", "licensePlate": "XX-99-ERR", "brand": "FAIL", "currentSpeed": 0}'
```

#### **Logs a Monitorizar:**

1. O fleet-service regista o veículo como PENDING e publica VehicleCreated.
2. O notification-service deteta a marca "FAIL" e emite VehicleSetupFailed com a razão do erro.
3. O fleet-service recebe o evento de erro de compensação e altera o estado de v-falha na base de dados PostgreSQL para CANCELLED.
4. O AuditConsumer emite um alerta vermelho de auditoria.

### **Cenário 3: Demonstração de Tracing Distribuído**

Verifique a consola do Docker Compose. Procure por qualquer operação correspondente a um dos pedidos cURL efetuados. O mesmo identificador de correlação (ex: [CorrelationID: a2b3-...]) estará impresso:

- No Express HTTP do fleet-service
- No evento processado pelo notification-service
- Nos logs de auditoria do AuditConsumer

### **Cenário 4: Demonstração do Circuit Breaker (Injeção de Falhas)**

1. Efetue um pedido de geração de relatório síncrono.  

```
curl -X POST http://localhost:3002/api/reports/generate \
-H "Content-Type: application/json" \
-d '{"vehicleId": "v-sucesso"}
```

*Resposta esperada:* 202 Accepted com um jobId. Aceda a `http://localhost:3002/api/reports/<jobId>` após alguns segundos para ver o relatório gerado com sucesso contendo o veículo Tesla obtido via gRPC.

2. **Derrube o Fleet Service:**  
`docker compose stop fleet-service`

3. **Submeta novos pedidos de relatórios:**  

```
curl -X POST http://localhost:3002/api/reports/generate \
-H "Content-Type: application/json" \
-d '{"vehicleId": "v-sucesso"}
```



Observe os logs do report-service. A chamada gRPC falhará imediatamente.

Após 3 tentativas falhadas (ou novos pedidos repetidos), observe a consola:

 [Circuit Breaker] O circuito abriu! As chamadas ao FleetService gRPC estão bloqueadas temporariamente.

Os pedidos seguintes falharão imediatamente sem qualquer espera de timeout no gRPC, ativando a DLQ e logs de erro apropriados.

4. **Recupere o Fleet Service:**

`docker compose start fleet-service`

Após o `resetTimeout` de 10s, envie um novo pedido de relatório. Verá o circuito transitar para `halfOpen`, testar o serviço e finalmente fechar (`close`), restabelecendo a operação normal e logs verdes de sucesso!